geo-distributed data centers

which are data centers located in different geographical regions across the world. In modern cloud computing, whenever a user accesses a web application such as Google, YouTube, or any cloud service, the request is handled by a server located in a data center. This communication depends heavily on the Internet, making it a critical component of application performance. Not only do users communicate with data centers, but data centers also communicate with each other. For example, user data is often replicated across multiple data centers to ensure reliability and availability. Therefore, wide-area network connectivity is just as important as the internal infrastructure of the data center itself.

Multiple data centers are required for several important reasons. First, they improve data availability. If one data center fails due to a natural disaster or technical issue, data can still be accessed from another location. Second, they enable load balancing by distributing incoming and outgoing traffic across different regions, preventing any single data center from becoming overloaded. Third, they reduce latency by bringing services closer to users geographically, which improves response time. Fourth, legal requirements in some countries mandate that data must be stored locally within that country. Finally, hybrid cloud operations allow companies to use private infrastructure for normal workloads and public cloud resources during peak demand.

Studies of inter-data center traffic, such as those conducted on Yahoo data centers, show that although the number of flows between data centers is only about 10% to 20% of client-to-data center traffic, these flows tend to last longer and carry significantly more data. This means inter-data center communication is heavy and critical. Additionally, these networks are unique because they use high-capacity dedicated links, often with bandwidths in the order of hundreds of gigabits per second, making them extremely expensive resources.

The traditional method used for managing traffic in such networks is called Multiprotocol Label Switching (MPLS). In MPLS, data is forwarded based on labels instead of full IP addresses, which simplifies routing. The

process begins with link-state protocols such as OSPF or IS-IS, which distribute information about the network topology so that every router has a complete map of the network. These protocols can also share information about available bandwidth on each link. When a new data flow arrives, the system selects a path with sufficient capacity and establishes a tunnel for that flow. MPLS also supports priority levels, allowing higher-priority traffic to displace lower-priority traffic if necessary. Once the path is established, only the ingress (entry) and egress (exit) routers examine the packet headers, while intermediate routers simply forward packets based on labels, making the process efficient.

Despite its advantages, MPLS has several limitations. One major problem is inefficiency. Networks are typically provisioned for peak traffic, but average utilization is much lower, leading to wasted bandwidth. Another issue is the lack of a global view of the network. Each router makes decisions independently based on local information, which results in suboptimal traffic engineering. Additionally, MPLS only provides fairness at the link level, not across the entire network, which can lead to unfair distribution of resources. Because of these limitations, such networks often operate at only around 30% utilization to maintain safety margins, which is costly.

To overcome these problems, modern systems such as Google's B4 and Microsoft's SWAN use Software Defined Networking (SDN). These systems rely on centralized control, where a central controller collects information about the entire network and makes optimized decisions about traffic routing. One key idea is to differentiate between types of traffic. Some applications, like video calls, are delay-sensitive, while others, like data backups, can tolerate delays. By prioritizing critical traffic and using flexible traffic to fill unused capacity, these systems achieve higher efficiency. They also dynamically reallocate bandwidth based on changing demands, often updating traffic allocations multiple times per day.

Google's B4 is a private wide-area network connecting its data centers globally. It uses centralized traffic engineering servers that gather information about network topology, available bandwidth, and traffic

demands, and then compute optimal routing decisions. These decisions are enforced through distributed controllers at each data center. B4 uses simple and cost-effective hardware but relies on sophisticated software to manage traffic. It also maintains backup routing using BGP as a fallback mechanism. The system achieves nearly 100% utilization and can compute traffic allocations very quickly.

Microsoft's SWAN system also focuses on efficient traffic engineering but emphasizes avoiding congestion during traffic updates. It carefully schedules changes in routing to ensure that no links become overloaded during transitions. Both B4 and SWAN demonstrate how centralized control and intelligent traffic management can significantly improve network performance compared to traditional approaches.

The second part of the lecture deals with time and clock synchronization in distributed systems. In a distributed environment, each machine has its own clock, and these clocks are not perfectly synchronized. This can lead to inconsistencies. For example, one server may record an event as happening before another server, even though the actual order was different. This can cause serious errors in applications such as booking systems. Therefore, synchronization is required to ensure correctness and fairness.

Two important concepts in this context are clock skew and clock drift. Clock skew refers to the difference in time between two clocks, while clock drift refers to the difference in their speeds. Even if clocks are initially synchronized, drift can cause them to diverge over time. Synchronization can be external, where clocks are aligned with a global standard like UTC, or internal, where clocks are synchronized with each other within a system.

Several algorithms are used for clock synchronization. Christian's algorithm involves a client requesting the current time from a server and adjusting its clock based on the round-trip time of the request. The Network Time Protocol (NTP) is widely used on the Internet and organizes time servers in a hierarchical structure, allowing clients to synchronize with higher-level servers. The Berkeley algorithm uses a master node that collects time values from all nodes, computes an average, and instructs

each node how to adjust its clock. The Datacenter Time Protocol (DTP) is a more advanced approach that uses hardware-level synchronization to achieve extremely high precision, often within nanoseconds. However, all these methods still have some error due to network delays.

Because perfect synchronization is difficult, distributed systems often use logical clocks instead of relying on physical time. Lamport clocks assign timestamps to events based on a simple counter. Each process increments its counter for each event and includes the timestamp in messages. When a message is received, the receiving process updates its clock using the maximum of its current value and the received timestamp, plus one. This ensures that causality is maintained, meaning if one event happens before another, its timestamp will be smaller. However, Lamport clocks cannot distinguish between events that are concurrent.

To address this limitation, vector clocks are used. In this approach, each process maintains a vector of counters, one for each process in the system. This allows the system to track not only the order of events but also whether two events are causally related or independent (concurrent). Vector clocks provide a more precise way of capturing relationships between events but require more storage and complexity.